

```

from google.colab import drive
drive.mount('/content/drive')

import os
import shutil
import numpy as np
import cv2
from tqdm import tqdm
from sklearn.model_selection import train_test_split, KFold
from sklearn.utils import shuffle
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dropout, Dense

```

Mounted at /content/drive

```

# ----- DATA SPLITTING -----
dataset_folder = "/content/drive/MyDrive/Top View Pics"
output_train_folder = "/content/drive/My Drive/output/train"
output_test_folder = "/content/drive/My Drive/output/test"
labels = ['AD Top View', 'CN Top View', 'MCI Top View']

os.makedirs(output_train_folder, exist_ok=True)
os.makedirs(output_test_folder, exist_ok=True)

for label in labels:
    section_path = os.path.join(dataset_folder, label)
    if not os.path.isdir(section_path): continue
    files = [f for f in os.listdir(section_path) if os.path.isfile(os.path.join(section_path, f))]
    train_files, test_files = train_test_split(files, test_size=0.2, random_state=42)
    train_section_path = os.path.join(output_train_folder, label)
    test_section_path = os.path.join(output_test_folder, label)
    os.makedirs(train_section_path, exist_ok=True)
    os.makedirs(test_section_path, exist_ok=True)
    for f in train_files: shutil.copy(os.path.join(section_path, f), os.path.join(train_section_path, f))
    for f in test_files: shutil.copy(os.path.join(section_path, f), os.path.join(test_section_path, f))

# ----- IMAGE LOADING -----
def load_images(folder_path, label, data_list, label_list):
    for image_file in tqdm(os.listdir(folder_path), desc=f>Loading {label}"):
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path)
        if image is not None:
            image = cv2.resize(image, (150, 150))
            data_list.append(image)
            label_list.append(label)

X_train, Y_train, X_test, Y_test = [], [], [], []
for label in labels:
    train_folder = os.path.join(output_train_folder, label)
    test_folder = os.path.join(output_test_folder, label)
    if os.path.exists(train_folder): load_images(train_folder, label, X_train, Y_train)
    if os.path.exists(test_folder): load_images(test_folder, label, X_test, Y_test)

X_train, Y_train = shuffle(np.array(X_train), np.array(Y_train), random_state=42)
X_test, Y_test = np.array(X_test), np.array(Y_test)

label_to_int = {label: i for i, label in enumerate(labels)}
Y_train_int = np.array([label_to_int[y] for y in Y_train])
Y_test_int = np.array([label_to_int[y] for y in Y_test])
Y_train_onehot = to_categorical(Y_train_int, 3)
Y_test_onehot = to_categorical(Y_test_int, 3)

```

Loading AD Top View: 100%  508/508 [00:05<00:00, 100.80it/s]
Loading AD Top View: 100%  128/128 [00:00<00:00, 135.07it/s]
Loading CN Top View: 100%  1075/1075 [00:14<00:00, 74.31it/s]
Loading CN Top View: 100%  269/269 [00:02<00:00, 120.80it/s]
Loading MCI Top View: 100%  1152/1152 [00:16<00:00, 70.03it/s]
Loading MCI Top View: 100%  288/288 [00:02<00:00, 109.35it/s]

```

# ----- MODEL DEFINITION (ResNet50) -----
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
x = GlobalAveragePooling2D()(base_model.output)

```

```

x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=output)

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# ----- TRAINING -----
history = model.fit(X_train, Y_train_onehot, validation_data=(X_test, Y_test_onehot), epochs=10, batch_size=32, verbose=1)

# ----- EVALUATION -----
test_loss, test_accuracy = model.evaluate(X_test, Y_test_onehot)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

pred_probs = model.predict(X_test)
pred_labels = np.argmax(pred_probs, axis=1)
actual_labels = np.argmax(Y_test_onehot, axis=1)

print(classification_report(actual_labels, pred_labels, target_names=['AD', 'CN', 'MCI']))

```

📄 Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels

```

94765736/94765736 3s 0us/step
Epoch 1/10
86/86 ----- 1170s 13s/step - accuracy: 0.6095 - loss: 1.0771 - val_accuracy: 0.2628 - val_loss: 23.0892
Epoch 2/10
86/86 ----- 1164s 13s/step - accuracy: 0.9162 - loss: 0.2583 - val_accuracy: 0.1898 - val_loss: 16.1492
Epoch 3/10
86/86 ----- 1169s 13s/step - accuracy: 0.9556 - loss: 0.1433 - val_accuracy: 0.1869 - val_loss: 26.3530
Epoch 4/10
86/86 ----- 1161s 13s/step - accuracy: 0.9830 - loss: 0.0403 - val_accuracy: 0.5066 - val_loss: 3.6667
Epoch 5/10
86/86 ----- 1161s 13s/step - accuracy: 0.9861 - loss: 0.0437 - val_accuracy: 0.8672 - val_loss: 0.4159
Epoch 6/10
86/86 ----- 1128s 13s/step - accuracy: 0.9791 - loss: 0.0532 - val_accuracy: 0.4686 - val_loss: 3.6464
Epoch 7/10
86/86 ----- 1128s 13s/step - accuracy: 0.9853 - loss: 0.0480 - val_accuracy: 0.8584 - val_loss: 0.5872
Epoch 8/10
86/86 ----- 1165s 13s/step - accuracy: 0.9960 - loss: 0.0148 - val_accuracy: 0.9372 - val_loss: 0.2724
Epoch 9/10
86/86 ----- 1147s 13s/step - accuracy: 0.9952 - loss: 0.0105 - val_accuracy: 0.5737 - val_loss: 3.7335
Epoch 10/10
86/86 ----- 1182s 13s/step - accuracy: 0.9724 - loss: 0.0763 - val_accuracy: 0.6774 - val_loss: 2.3303
22/22 ----- 58s 3s/step - accuracy: 0.6818 - loss: 2.3615
Test Loss: 2.3303
Test Accuracy: 67.74%
22/22 ----- 64s 3s/step

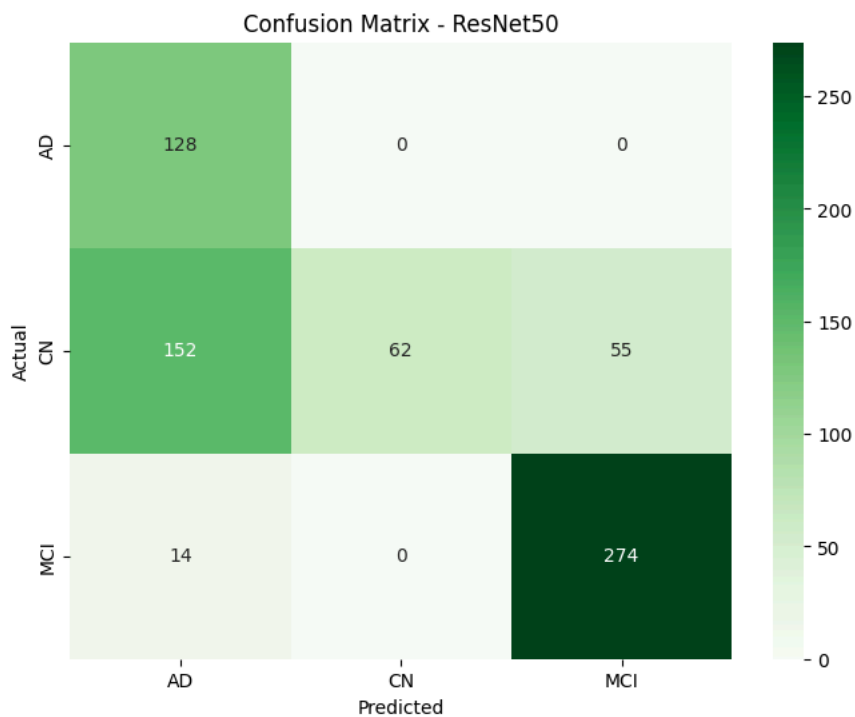
```

	precision	recall	f1-score	support
AD	0.44	1.00	0.61	128
CN	1.00	0.23	0.37	269
MCI	0.83	0.95	0.89	288
accuracy			0.68	685
macro avg	0.76	0.73	0.62	685
weighted avg	0.82	0.68	0.63	685

```

# ----- CONFUSION MATRIX (Green) -----
cm = confusion_matrix(actual_labels, pred_labels)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens', xticklabels=['AD', 'CN', 'MCI'], yticklabels=['AD', 'CN', 'MCI'])
plt.title("Confusion Matrix - ResNet50")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.savefig('confusion_matrix_green_resnet.png', dpi=300)
plt.show()

```

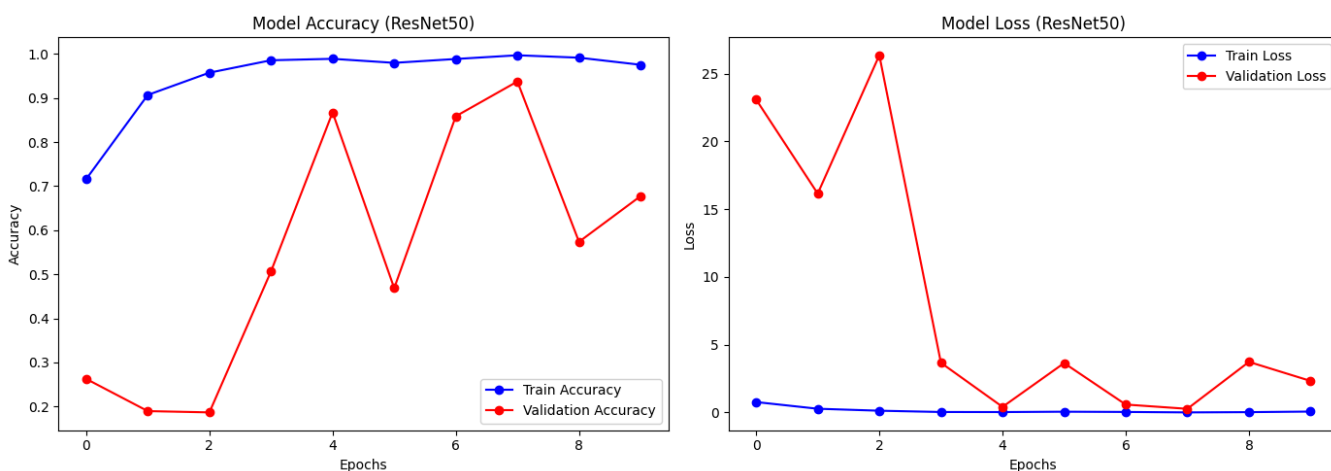


```
plt.figure(figsize=(14, 5))
```

```
# Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], 'bo-', label='Train Accuracy')
plt.plot(history.history['val_accuracy'], 'ro-', label='Validation Accuracy')
plt.title('Model Accuracy (ResNet50)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

```
# Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], 'bo-', label='Train Loss')
plt.plot(history.history['val_loss'], 'ro-', label='Validation Loss')
plt.title('Model Loss (ResNet50)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

```
plt.tight_layout()
plt.savefig('accuracy_loss_resnet.png', dpi=300)
plt.show()
```



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.